Component embodiment

- process to create executable, deployable component from abstract spec.

- system – abstract component
  - ↳ decompose to lower level components
  - ↳ concretize
  - ↳ runnable binary codes
  - (not linear) -> 3D model
- automatize embodiment steps
  - ↳ create source code → binary code
  - ↳ we need human resource
    - ↳ implementation decisions → the mode and result can be different
    - ↳ manual work
    - ↳ goal: model
      - ↳ ~~not exa~~ not mandatory the 100%
  - ↳ problem: some decisions not documented

SORT technique

- Systematic Object-Oriented Refinement and Translation
- separate the refinement and translation
- use NOF (Normal Obj. Form)
  - minimize the gap between object oriented model and program

Refinement

- same thing in a more detailed other way

Complation

- the two relations are diff things
  - ↳ the detailness is same

Steps

1. · refine to pre-defined models
2. · translate it into source code

SCART adv.

- easier embodiment
  - ↳ small steps are needed
- separted problems
  - ↳ only one task is in focus
  - ↳ easier refinement
- identify and use similarities
  - · multiple impl. are supported
  - · new impl is hard task

Re-using components

not fully met the requirement (spec)
· its connected to the:
  · composition / decomposition } two dimensions
  · abstraction / concretization

↳
↳ change model to the component ↓ (against comp. based dev)
↳ rewrite component ↓ (not possible)
↳ adaptation component
(wrapper, bridge)

Create new comp.

- if not exist
- more decomposition
  - ↳ may be there are available components for the smaller parts

No instructions for logical and physical components and their installation

Product Family concepts

- the decision model are not easy
  - what can be product family
  - ↳ 3 defined items
    - ↳ textual question about the decision
    - ↳ possible answers
    - ↳ the decision place
- all product in the family is supported by decision models
  - ↳ for models:
    - ↳ environment map
    - ↳ comp spec
    - ↳ comp realization
  - ↳ the engineer should give answers to the questions

Framework engineering

- the core ~~chose~~ in a prd. family
- same plan, model and impl systems like for sys dev

- its summary of componement that not fully completed → more components are needed to finish
- the unge sys is an entity of a generic sys

Quality assurance requirements

- define the meaning the quality
- define what quality aspect are important and how to ~~us~~ ensure them during development

Documentation

can be useful
- not as abstract as like the specs
- deeper view of the comp. using
- useful: define the sequences and combinations of the methods and effects

Summary

- need a ~~D~~ methodology (like Kobra)
- Kobra
  - ↳ important istructions
  - ↳ using of UML in comp-based dev in a project

Component based dev

Composition mechanisms

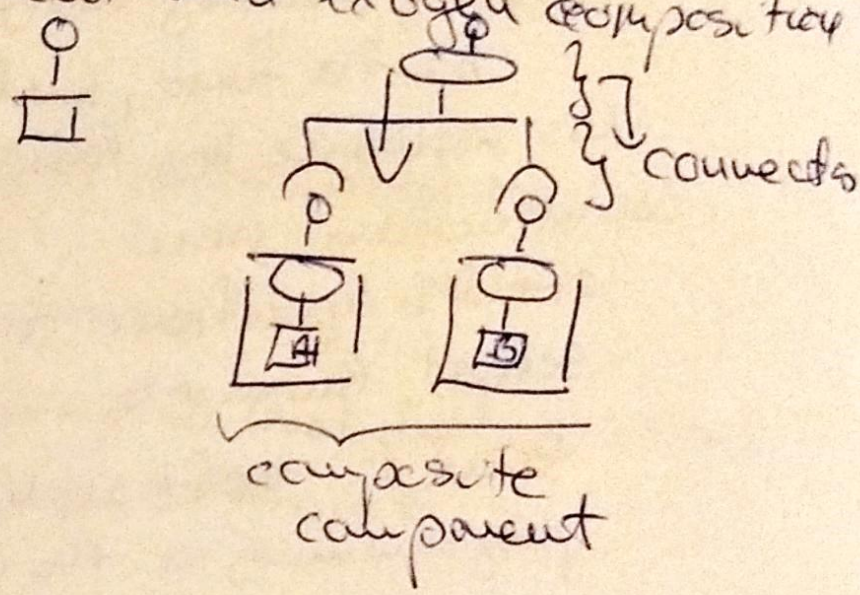
- integrate software constructions
  - ↳ components, re-using, systematic construction
- automatizable, systematic

- diff composition units
- composition mechanism creates complex behaviour from behaviour
- different views
  - programming view
    - ↳ programmer do
      - ↳ create programer app from parts
    - ↳ programming lang
      - ↳ ~~fa~~ func, method, classes,
    - ↳ sequence by lang. programming in the small
  - construction view
    - creates app from connection of components
    - script languages
    - ~~re~~sult: soft arch.
    - programming in the large
  - CBD view
    - extend construction view with comp. models and re-using
    - composition units: objects, arch. units

CBD view

- objects have service interface but not have require interface
- the arch entities have input and output ports
- encapsulated component: has service interface, not have require interface
  - ↳ data and control → not call other components

- general components connects through service and require interfaces
- objects can't connect, because they don't have require interface
  - ↳ connect with method calls
- the arch entities connect with compatible ports
- encapsulated component need exogen composition connector



Composition mechanisms

- include, extend, connect, coordination
- Include (contained)
  - ↳ the units get in the composite unit
  - ↳ the composite behaviour is the units' behaviour together
  - ↳ composition (lifecycle)
    - ↳ the contained units are managed by the composit
  - ↳ aggregation (lifecycle)
    - ↳ the contained units are ~~manage~~ independence from the composit

- Extension
  - ↳ the composit behaviour extends the units behavior (like multiple inheritance)
    - ↳ we create a class from 2 others
  - ↳ only for classes (not for aspects)
- Connection
  - composit behavior is the units behaviour that are connected
  - the units calls each other's behaviour
  - direct message
    - ↳ direct method calls (delegation)
  - indirect message
    - ↳ through connectors
  - ↳ only for components (not for classes)
    - ↳ require, service interfaces
    - ↳ assembly
    - ↳ port forwarding (delegation)
- Coordination
  - the behaviour of the composit is the coordination of the ~~component~~ units
  - ↳ coordination by a coordinator
    - ↳ throug data or control channel with the units
    - ↳ the units not call eachother (seperated units)
  - ↳ pl: asyuc process coordination